

PENETRATION TESTING ASSESSMENT REPORT

DVWA — Reflected Cross-Site Scripting (XSS)

Prepared by:	Muhammad Adeel Tariq
Certification:	CEH (EC-Council) Certified
Training Program:	NAVTTTC-funded Cybersecurity Training, Corvit Institute, Rawalpindi
Classification:	Educational / Controlled Lab Environment

EXECUTIVE SUMMARY

A Reflected Cross-Site Scripting (XSS) vulnerability was identified and successfully exploited on DVWA (Damn Vulnerable Web Application) running on Metasploitable 2. The vulnerability allows an attacker to inject malicious JavaScript that executes in the victim's browser, potentially leading to session hijacking, credential theft, and account takeover. **Severity: HIGH.**

SCOPE

Target IP Address:	192.168.56.102
Target Application:	DVWA (Damn Vulnerable Web Application)
Security Level Tested:	High
Testing Environment:	Isolated Lab Environment

Practical :

Input `<script>alert(1)</script>` reflects back. Vulnerability name and impact?

Step-by-Step Testing Procedure:

Step 1: Target Identification

The first step was to identify the IP address of the Metasploitable 2 virtual machine. The `ifconfig` command was executed on the target system, revealing the IP address 192.168.56.102. This confirmed the target was running and accessible on the network.

```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:2c:87:aa
          inet addr:192.168.56.102  Bcast:192.168.56.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe2c:87aa/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:3 errors:0 dropped:0 overruns:0 frame:0
          TX packets:30 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1252 (1.2 KB)  TX bytes:3924 (3.8 KB)
          Base address:0xd020  Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:103 errors:0 dropped:0 overruns:0 frame:0
          TX packets:103 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:23757 (23.2 KB)  TX bytes:23757 (23.2 KB)

msfadmin@metasploitable:~$
```

Step 2: Accessing the Web Application

Using a web browser on the attacker machine (Kali Linux), the Damn Vulnerable Web App was accessed at `http://192.168.56.102/dvwa/login.php`. The login page loaded successfully, confirming the web server was operational.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Welcome to Damn Vulnerable Web App!

Damn Vulnerable Web App (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goals are to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and aid teachers/students to teach/learn web application security in a class room environment.

WARNING!

Damn Vulnerable Web App is damn vulnerable! Do not upload it to your hosting provider's public html folder or any internet facing web server as it will be compromised. We recommend downloading and installing [XAMPP](#) onto a local machine inside your LAN which is used solely for testing.

Disclaimer

We do not take responsibility for the way in which any one uses this application. We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

General Instructions

The help button allows you to view hits/tips for each vulnerability and for each security level on their respective page.

You have logged in as 'admin'

Step 3: Navigating to the XSS Reflected Page

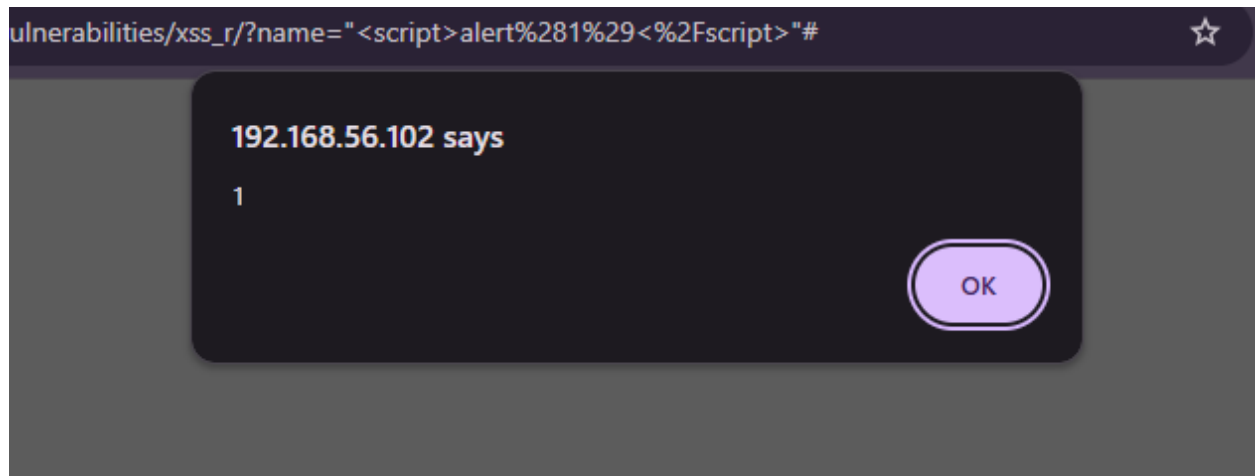
From the main menu, the "XSS reflected" option was selected. This page accepts user input through a text field labeled "What's your name?" and reflects the input back to the browser without any filtering or encoding.

Step 4: Payload Injection

A standard XSS proof-of-concept payload was entered into the input field. The payload used was: `<script>alert(1)</script>`

Step 5: Vulnerability Confirmation

After submitting the payload, the application responded by displaying "Hello" followed by the injected script. Instead of treating the script as plain text, the browser interpreted it as executable JavaScript code. As a result, an alert dialog box appeared on the screen displaying the number "1".



Remediation Recommendations

- **Output Encoding** – Use `htmlspecialchars()` to convert `<` into `<` and `>` into `>`
- **Input Validation** – Allowlist only safe characters (letters, numbers), reject everything else
- **Content Security Policy (CSP)** – Add `Content-Security-Policy: script-src 'self'` header to block inline scripts
- **HttpOnly Cookie Flag** – Set `HttpOnly` attribute on session cookies to prevent JavaScript access
- **X-XSS-Protection Header** – Add `X-XSS-Protection: 1; mode=block` for browser-based filtering
- **Regular Security Testing** – Perform routine XSS scans using tools like OWASP ZAP or Burp Suite

What I Learned

The application reflected user input directly into the page without any output encoding, allowing a script tag to be interpreted and executed by the browser rather than displayed as text. This reinforced that user-supplied input must never be trusted or rendered as-is — even a single unescaped field is enough to allow arbitrary JavaScript execution in a victim's session.

Vulnerability Classification

CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-Site Scripting)

Environment Disclaimer

This assessment was performed against DVWA running on an isolated Metasploitable 2 instance inside a controlled lab environment. No unauthorized systems were accessed.